

Parcial - tema1

Nota: 7.9 / 10.0 (APROBADO)

puntaje ej1: 0.4
puntaje ej2: 0.0
puntaje ej3: 0.5
puntaje ej4: 2
puntaje ej5: 2
puntaje ej6: 2
puntaje ej7: 0.5
puntaje ej8: 0.5

Datos del alumno

Nombre: Walter Fernando Fregenal

DNI: 22466445

LU:

Maquina: 62-06

Enunciado

Parcial Haskell - Tema 1 - Turno mañana

Importante

Template de funciones a implementar [acá](#)

Lista de funciones permitidas [acá](#)

Ejemplo de hunit [acá](#)

Enunciado

Resolver los siguientes ejercicios cuyas especificaciones en lenguaje semiformal figuran a continuación. Deben ser implementadas en Haskell utilizando los tipos requeridos y solamente las funciones que se ven en la materia Introducción a la Programación / Algoritmos y Estructuras de Datos I (FCEyN-UBA).

Ejercicio 1 [2 puntos]

problema ejercicio1 (*s*: seq(Z), *n*: Z) : seq(Z) {
 requiere: {existe un índice *i* tal que $0 \leq i < |s|-1$ y $s[i]+s[i+1] = n$ }
 asegura: {|*res*| = |*s*| - 2}
 asegura: {*res* es igual a *s* sin *s*[*j*] ni *s*[*j*+1], donde *j* es una posición válida de *s* tal que $s[j] + s[j+1] = n$ }
}

Ejercicio 2 [0,5 puntos]

Conteste marcando la opción correcta.

¿Qué nombre le pondrías a la función del ejercicio anterior?

- ☐ SacarTodosLosParesConsecutivosQueSumanN
- ☐ SacarAlgunParConsecutivoQueSumaN
- ☐ SacarPrimerParConsecutivoQueSumaN
- ☐ Ninguna de las opciones anteriores describe adecuadamente el problema

Ejercicio 3 [0,5 puntos]

Conteste marcando la opción correcta, teniendo el cuenta el problema ejercicio1

Si s = [-3, 0, -1, 4, -2] y n = -3 , entonces:

- ☐ res1 = [-1, 4, -2] es una salida válida
- ☐ res2 = [-2] es una salida válida
- ☐ No es posible ejecutar la función con un valor negativo para *n*
- ☐ No hay problema con ejecutar la función para un valor negativo de *n* , pero res1 y res2 no son soluciones válidas de acuerdo a la especificación

Ejercicio 4 [2 puntos]

Un número entero positivo es **refactorizable** si es divisible por la cantidad de divisores positivos que tiene (contando el 1 y al propio número). Por ejemplo, 9 tiene como divisores a 1, 3 y 9 (3 divisores en total), y 9 es divisible por 3, por lo tanto 9 es refactorizable. En cambio, 10 tiene como divisores a 1, 2, 5 y 10 (4 divisores), y 10 no es divisible por 4, por lo tanto no es refactorizable.

problema cantidadDeRefactorizables (*d*: Z, *h*: Z) : Z {
 requiere: { $0 < d \leq h$ }
 asegura: {*res* es la cantidad de números refactorizables en el rango [*d*..*h*]}
}

Ejemplo: cantidadDeRefactorizables 1 10 debe devolver 4
(porque 1, 2, 8 y 9 son refactorizables en ese rango)

Ejercicio 5 [2 puntos]

Representaremos una compra con una tupla *String* x *Z* x *Bool*, donde:

- La primera componente de la tupla contiene el nombre del cliente
- La segunda componente de la tupla contiene el monto de la compra
- La tercera componente de la tupla indica si el cliente es socio del club de fidelidad

Se pide implementar *sociosConComprasAltas*, que dada una lista de compras y un monto umbral, devuelva los nombres de los socios que realizaron al menos una compra por un monto mayor al umbral.

problema sociosConComprasAltas (*compras*: seq(String x Z x Bool), *umbral*: Z) : seq(String) {
 requiere: {*umbral* ≥ 0}
 requiere: {|*compras*[*i*]| ≥ 0 para todo *i* tal que $0 \leq i < |compras|$ }
 asegura: {*res* no tiene elementos repetidos}
 asegura: {*res* contiene los nombres de todos los clientes incluidos en *compras* tales que son socios y realizaron al menos una compra con monto estrictamente mayor a *umbral*}
 asegura: {*res* contiene solamente los nombres de clientes que cumplen la condición anterior}
}

Ejercicio 6 [2 puntos]

En Haskell, una matriz se puede representar utilizando una secuencia de secuencias, donde cada secuencia interna representa una fila de la matriz. Todas las filas deben tener igual longitud.

Implementar la siguiente función:

problema ejercicio6 (*matriz*: seq(seq(Z))) : seq(seq(Z)) {
 requiere: {|*matriz*| > 0}
 requiere: {para toda fila perteneciente a *matriz*, |*fila*| = |*matriz*|}
 asegura: {|*res*| = |*matriz*|}
 asegura: {para todo *i* tal que $0 \leq i < |matriz|$, |*res*[*i*] = |*matriz*[*i*]|}
 asegura: {para todo *i*, *j* tales que $0 \leq i < |matriz|$ y $0 \leq j < |matriz|$: si $i = j$ entonces $res[i][j] = 0$, si no $res[i][j] = matriz[i][j]$ }
}

Ejercicio 7 [0,5 puntos]

Conteste marcando la opción correcta.

¿Qué nombre le pondrías a la función del ejercicio anterior?

- ☐ AnularDiagonal
- ☐ AnularMatriz
- ☐ ObtenerDiagonal
- ☐ Ninguna de las opciones anteriores es un buen nombre para la función

Ejercicio 8 [0,5 puntos]

Tenemos una especificación cuyo **requiere** indica $n \neq 0$. Creamos un test suite con 3 casos:

- **caso1:** input *n* = 0
- **caso2:** input *n* = -2
- **caso3:** input *n* = 5

Al ejecutar los casos de test obtenemos 1 falla.

¿Cuál de las siguientes opciones puede afirmarse con seguridad?

- ☐ El programa tiene un bug
- ☐ El test suite no está bien diseñado
- ☐ Ambas son ciertas (el programa tiene un bug y además el test suite no están bien diseñado)

Adjunta el archivo con tu solución:

Solo se puede adjuntar 1 archivo de extensión .hs. En caso de haber desarrollado tests propios, no deben ser entregados.

Seleccionar archivo a enviar: Ningún archivo seleccionado

Enviar

Solucion entregada por el alumno

```
module SolucionT1 where

-- Ejercicio 1
ejercicio1 :: [Integer] -> Integer -> [Integer]
ejercicio1 [] _ = []
ejercicio1 (actual:siguiente:xs) numero
```

```
| (actual + siguiente) == numero = xs
| otherwise = actual : ejercicio1 xs numero

-- Ejercicio 4
cantidadDeRefactorizables :: Integer -> Integer -> Integer
cantidadDeRefactorizables indice limite
| indice > limite = 0
| numRefactorizable indice = 1 + (cantidadDeRefactorizables (indice + 1) limite)
| otherwise = cantidadDeRefactorizables (indice + 1) limite

numRefactorizable :: Integer -> Bool
numRefactorizable 1 = True
numRefactorizable numero
| mod numero (auxCantidadDivisores numero) == 0 = True
| otherwise = False

auxCantidadDivisores :: Integer -> Integer
auxCantidadDivisores 1 = 1
auxCantidadDivisores n = sumalista(auxDivisoresHasta 1 n)

sumalista :: [Integer] -> Integer
sumalista [] = 0
sumalista (x:xs) = 1 + sumalista xs

auxDivisoresHasta :: Integer -> Integer -> [Integer]
auxDivisoresHasta 1 1 = [1]
auxDivisoresHasta indice numero
| indice > numero = []
| mod numero indice == 0 = indice : auxDivisoresHasta (indice+1) numero
| otherwise = auxDivisoresHasta (indice+1) numero

-- Ejercicio 5

type Cliente = String
type Monto = Integer
type Socio = Bool
type Compra = (Cliente, Monto,Socio)
type Compras = [Compra]
type Umbral = Integer

sociosConComprasAltas :: [(String, Integer, Bool)] -> Integer -> [String]
sociosConComprasAltas [] _ = []
sociosConComprasAltas compras umbral = quitoRepetidos (auxListaSociosConComprasAltas compras umbral)

auxListaSociosConComprasAltas :: Compras -> Umbral -> [Cliente]
auxListaSociosConComprasAltas [] _ = []
auxListaSociosConComprasAltas ((cliente, monto, socio):xs) umbral
| socio == False = auxListaSociosConComprasAltas xs umbral
| socio == True && monto > umbral = cliente : auxListaSociosConComprasAltas xs umbral
| otherwise = auxListaSociosConComprasAltas xs umbral

quitoRepetidos :: [Cliente] -> [Cliente]
quitoRepetidos [] = []
quitoRepetidos (clientetop:xs)
| contieneCliente clientetop xs = quitoRepetidos xs
| otherwise = clientetop : quitoRepetidos xs

contieneCliente :: Cliente -> [Cliente] -> Bool
contieneCliente _ [] = False
contieneCliente clientetop (x:xs)
| clientetop == x = True
| otherwise = contieneCliente clientetop xs

-- Ejercicio 6
ejercicio6 :: [[Integer]] -> [[Integer]]
--ejercicio6 _ = []
ejercicio6 matriz = modificoMatriz 0 0 0 matriz

type Lista = [Integer]
type Indice = Integer
type Contador = Integer
type Numero = Integer
type Longitud = Integer
type Matriz = [Lista]

modificoMatriz :: Integer -> Indice -> Contador-> [[Integer]] -> [[Integer]]
modificoMatriz _ _ _ [] = []
modificoMatriz numero indice contador (fila : xs)
| contador < longitud = iesimoElemMod numero indice fila : modificoMatriz numero (indice + 1) (contador + 1) xs
| otherwise = []
where longitud = longLista fila

longLista :: [Integer] -> Integer
longLista [] = 0
longLista (x:xs) = 1 + longLista (xs)

iesimoElemMod :: Numero -> Indice -> Lista -> Lista
iesimoElemMod _ _ [] = []
iesimoElemMod numero indice lista
| indice < longitud = auxIesimo numero indice 0 lista
| otherwise = []
where longitud = longLista lista
auxIesimo :: Numero -> Indice -> Contador-> Lista -> Lista
auxIesimo _ _ _ [] = []
auxIesimo numero indice contador (actual:xs)
| indice == contador = numero : xs
| otherwise = actual : auxIesimo numero indice (contador+1) xs
```

Resultado de la compilacion

```
Loaded package environment from /home/jgodoy/.ghc/x86_64-linux-9.4.7/environments/default
[1 of 1] Compiling SolucionT1          ( correcciones/parcial_funcional_tm/fregenal_walter_fernando_22466445_X/submission.hs.main.hs, correcciones/parcial_funcional_tm/fregenal_walter_fernando_22466445_X/submission.hs.main.o )
```

Ejecucion de los tests

tema1-test-ej1.hs.compilacion.out

Puntaje del ej: 0.4 / 2

```
### Error in: 0:Ejemplo simple, un unico par valido
Solucion.hs:(6,1)-(9,47): Non-exhaustive patterns in function ejercicio1
```

Cases: 8 Tried: 1 Errors: 1 Failures: 0

Cases: 8 Tried: 2 Errors: 1 Failures: 0

Cases: 8 Tried: 3 Errors: 1 Failures: 0

```
### Error in: 3:Tres posiciones validas dispersas
Solucion.hs:(6,1)-(9,47): Non-exhaustive patterns in function ejercicio1
```

Cases: 8 Tried: 4 Errors: 2 Failures: 0

Cases: 8 Tried: 5 Errors: 2 Failures: 0

Cases: 8 Tried: 6 Errors: 2 Failures: 0

```
### Error in: 6:n negativo con multiples posiciones validas
Solucion.hs:(6,1)-(9,47): Non-exhaustive patterns in function ejercicio1
```

Cases: 8 Tried: 7 Errors: 3 Failures: 0

```
### Failure in: 7:Suma cero con multiples posiciones validas
Tema1-test-ej1.hs:28
Se esperaba una de estas soluciones: [[0,3,-3,8], [0,5,-5,8]] . Se obtuvo [0,-5,-3]
```

Cases: 8 Tried: 8 Errors: 3 Failures: 1

Cases: 8 Tried: 8 Errors: 3 Failures: 1

mchoice.json-ej2.compilacion.out

Puntaje del ej: 0.0 / 0.5

mchoice ej2: respuesta del alumno=3, respuesta correcta=2

Ran 1 test in 0 seconds

FAILED (failures=1)

mchoice.json-ej3.compilacion.out

Puntaje del ej: 0.5 / 0.5

mchoice ej3: respuesta del alumno=1, respuesta correcta=1

Ran 1 test in 0 seconds

OK

tema1-test-ej4.hs.compilacion.out

Puntaje del ej: 2 / 2

Cases: 9 Tried: 1 Errors: 0 Failures: 0

Cases: 9 Tried: 2 Errors: 0 Failures: 0

Cases: 9 Tried: 3 Errors: 0 Failures: 0

Cases: 9 Tried: 4 Errors: 0 Failures: 0

Cases: 9 Tried: 5 Errors: 0 Failures: 0

Cases: 9 Tried: 6 Errors: 0 Failures: 0

Cases: 9 Tried: 7 Errors: 0 Failures: 0

Cases: 9 Tried: 8 Errors: 0 Failures: 0

Cases: 9 Tried: 9 Errors: 0 Failures: 0

Cases: 9 Tried: 9 Errors: 0 Failures: 0

tema1-test-ej5.hs.compilacion.out

Puntaje del ej: 2 / 2

Cases: 12 Tried: 1 Errors: 0 Failures: 0

Cases: 12 Tried: 2 Errors: 0 Failures: 0

Cases: 12 Tried: 3 Errors: 0 Failures: 0

Cases: 12 Tried: 4 Errors: 0 Failures: 0

Cases: 12 Tried: 5 Errors: 0 Failures: 0

Cases: 12 Tried: 6 Errors: 0 Failures: 0

Cases: 12 Tried: 7 Errors: 0 Failures: 0

Cases: 12 Tried: 8 Errors: 0 Failures: 0

Cases: 12 Tried: 9 Errors: 0 Failures: 0

Cases: 12 Tried: 10 Errors: 0 Failures: 0

Cases: 12 Tried: 11 Errors: 0 Failures: 0

Cases: 12 Tried: 12 Errors: 0 Failures: 0

Cases: 12 Tried: 12 Errors: 0 Failures: 0

tema1-test-ej6.hs.compilacion.out

Puntaje del ej: 2 / 2

Cases: 10 Tried: 1 Errors: 0 Failures: 0

Cases: 10 Tried: 2 Errors: 0 Failures: 0

Cases: 10 Tried: 3 Errors: 0 Failures: 0

Cases: 10 Tried: 4 Errors: 0 Failures: 0

Cases: 10 Tried: 5 Errors: 0 Failures: 0

Cases: 10 Tried: 6 Errors: 0 Failures: 0

Cases: 10 Tried: 7 Errors: 0 Failures: 0

Cases: 10 Tried: 8 Errors: 0 Failures: 0

Cases: 10 Tried: 9 Errors: 0 Failures: 0

Cases: 10 Tried: 10 Errors: 0 Failures: 0

Cases: 10 Tried: 10 Errors: 0 Failures: 0

mchoice.json-ej7.compilacion.out

Puntaje del ej: 0.5 / 0.5

mchoice ej7: respuesta del alumno=1, respuesta correcta=1

Ran 1 test in 0 seconds

OK

mchoice.json-ej8.compilacion.out

Puntaje del ej: 0.5 / 0.5

mchoice ej8: respuesta del alumno=2, respuesta correcta=2

Ran 1 test in 0 seconds

OK

FIN